

✓ ML/AI Project : Model Comparison vs Live Test

Background

With the advancement of deep learning and the emergence of technologies such as autonomous vehicles and in-game hit prediction, a question arises: is it possible to achieve both an efficient model and one that is accurate and comprehensive? In the field of computer vision, particularly in object detection, it is exceedingly challenging to identify a model that simultaneously excels in accuracy, robustness, and efficiency. Typically, a compromise is required.

To assess this trade-off between speed and detection efficacy, computer vision models are typically evaluated using three primary metrics: operational latency, mean average precision at an IoU threshold of 0.5, and recall. Operational latency denotes inference speed, while mAP50 quantifies overall detection performance by assessing precision and recall. Recall represents the proportion of actual objects that are successfully detected. When an architecture is reduced in scale to decrease latency—for instance, transitioning from a YOLO11-small model to a YOLO11-nano model—the reduced model capacity and computational demands frequently result in lower mAP50 and recall, although the precise effect is contingent upon the dataset and training parameters.

Recent literature explores methods for mitigating the trade-off between computational efficiency and detection performance. Fan, Q. et al. (2025) addressed this challenge through the development of lightweight architectures that reduce computational complexity using sparse attention mechanisms. Their findings suggest that, depending on the task, full model capacity is not always necessary. This allows smaller, more efficient models to maintain competitive performance when the underlying data exhibit sufficient structural regularity. Additionally, the operational environment or context typically influences the practicality of model simplification. For instance, real-world scenarios, particularly involving visual data, are often complex and disordered. Blackadar et al. (2021) highlight that geospatial imagery frequently exhibits substantial background variability and visual interference, necessitating greater computational power for accurate object identification. Similarly, Lu and Bie (2025) utilised a semantic prompt segmentation method to enhance target localisation in complex and poorly defined visual scenes (also referred to as unstructured imagery).

While traditional research frequently focuses on mitigating the visual complexity of real-world imagery, stylised synthetic environments offer a useful context for examining the balance between computational efficiency and detection performance. For instance, a virtual world such as Minecraft allows for a clear and controlled comparison between computational efficiency and detection performance. Within a voxel-based environment such as Minecraft, objects are typically defined by orthogonal geometry, simplified textures, and reduced visual uncertainty. Wang et al. (2025) argue that deep learning models should be evaluated based on both performance and efficiency, adapted to specific deployment scenarios rather than relying solely on generalised benchmark criteria.

To investigate this concept, this project compares two object detection models from Ultralytics: YOLO11n (Nano) and YOLO11s (Small). The models are evaluated using mean average precision, recall, and inference latency on the public Minecraft mobs dataset published by Two Turtles (2026) on Hugging Face. Following quantitative evaluation, an independent qualitative assessment is conducted using a local JPG image containing a Minecraft mob to determine whether observed performance trends are reflected in a practical inference scenario.

Throughout this document, we compare two YOLO models from Ultralytics: YOLO11n (Nano) and YOLO11s (Small). We evaluate their mAP50, recall, and latency. Subsequently, a test is conducted using a JPG image featuring a Minecraft mob to determine whether the statistical results align with a Minecraft mob inference scenario.

✓ Method

This section details the methodology employed for the implementation, training, and evaluation of the YOLO11n and the YOLO11s object detection models. It begins with a description of the dataset collection and preparation process, followed by an explanation of the model's implementation and training procedures. Subsequently, the evaluation methods used to determine the model's performance metrics are presented. Finally, the section concludes with an analysis of the results and a test using a JPEG image of a Minecraft model to demonstrate how the model's practical performance aligns with the statistical findings.

✓ Section 1: Imports and Dataset Collection

The process begins by downloading and updating the required external toolkits, including Ultralytics and Hugging Face Hub. Python modules are then imported to handle file system navigation (os, shutil), JSON file processing (json), data splitting (scikit-learn), and data organisation (pandas). Ultralytics is used to import the YOLO models, while Hugging Face is used to access the dataset. The !wget command is used to download the JPEG test image of the Minecraft mob. Following this, the training and validation folder structure is created. The annotation file is then opened, after which the images are shuffled and divided into training and validation sets.

```
#####
```

```
#section 1: Imports and dataset collection
#####

#Install libraries
!pip install -q ultralytics huggingface_hub scikit-learn
import os
import json
import shutil
from collections import defaultdict
from sklearn.model_selection import train_test_split
#Import tools
from collections import Counter
import matplotlib.pyplot as plt
import pandas as pd
from IPython.display import Image, display
#Import Model from Ultralytics
from ultralytics import YOLO
#Set data path and downloads dataset from Huggingface
from huggingface_hub import snapshot_download
DATASET_PATH = "/content/minecraft_dataset"

repo_dir = snapshot_download(
    repo_id="twoturtles/minecraft-mobs",
    repo_type="dataset"
)

print(repo_dir)

# Downloads jpg test image from github
!wget -O test_image.jpg "https://raw.githubusercontent.com/Magistrate-dot/ML-AI-project-_Minecraft_Object_Detection/main/Test"
```

Show hidden output

Section 2 : data Preparation

Following this, the training and validation folder structure is created. The annotation file is then opened, after which the images are shuffled and divided into training and validation sets.

Next, the raw pixel coordinates are converted into percentage values and stored as text files. These files are named to match their corresponding image files, with the specific mob classification IDs stored within them, forming the bounding box annotations used by the object detection models. A blueprint file (data.yaml) is then created to specify the locations of the images and their corresponding bounding box annotations. Finally, to verify that all categories correctly correspond to their assigned ID numbers and mob names, an explicit tally is compiled, and a breakdown table is printed showing the exact number of individual chickens, cows, creepers, and other mobs present throughout the dataset.

```
#####
#Section 2: Data preparation
#####

#Creating folder structure
for split in ["train", "valid"]:
    os.makedirs(f"{DATASET_PATH}/{split}/images", exist_ok=True)
    os.makedirs(f"{DATASET_PATH}/{split}/labels", exist_ok=True)

#Opens annotations file, then shuffles images and splits them
with open(os.path.join(repo_dir, "annotations.json"), "r") as f:
    coco = json.load(f)

images = coco["images"]
annotations = coco["annotations"]

annotations_by_image = defaultdict(list)

for ann in annotations:
    annotations_by_image[ann["image_id"]].append(ann)

train_images, valid_images = train_test_split(
    images,
    test_size=0.20,
    random_state=42,
    shuffle=True
)

print("Training images:", len(train_images))
print("Validation images:", len(valid_images))

#This section does two parts.
#1. Copies raw image files into the new image folder.
```

```

def process_split(image_list, split_name):

    for img in image_list:

        image_id = img["id"]
        filename = img["file_name"]

        src = os.path.join(repo_dir, "images", filename)
        dst = os.path.join(DATASET_PATH, split_name, "images", filename)

        shutil.copy(src, dst)

        label_file = os.path.join(
            DATASET_PATH,
            split_name,
            "labels",
            filename.replace(".png", ".txt")
        )
    #2. Converts pixel coordinates into percentages
    # then writes a new label file containing the category number for the mob and its co--ordinates
    with open(label_file, "w") as f:

        for ann in annotations_by_image.get(image_id, []):

            x, y, w, h = ann["bbox"]

            xc = (x + w / 2) / img["width"]
            yc = (y + h / 2) / img["height"]
            wn = w / img["width"]
            hn = h / img["height"]

            cls = ann["category_id"]

            f.write(
                f"{cls} {xc} {yc} {wn} {hn}\n"
            )
process_split(train_images, "train")
process_split(valid_images, "valid")

# Generates the file named data.yaml
yaml_text = f"""
path: {DATASET_PATH}

train: train/images
val: valid/images

names:
  0: chicken
  1: cow
  2: creeper
  3: enderman
  4: pig
  5: sheep
  6: skeleton
  7: spider
  8: zombie
"""

with open("data.yaml", "w") as f:
    f.write(yaml_text)

# Summary verification, prints a final count of the files
print("Train images:", len(os.listdir(f"{DATASET_PATH}/train/images")))
print("Train labels:", len(os.listdir(f"{DATASET_PATH}/train/labels")))

print("Validation images:", len(os.listdir(f"{DATASET_PATH}/valid/images")))
print("Validation labels:", len(os.listdir(f"{DATASET_PATH}/valid/labels")))

# Count annotations per class
class_counts = Counter()

# Tally the the the category ID's for each mob
for ann in annotations:
    class_counts[ann["category_id"]] += 1

# Class names and maps the ID numbers to names
class_names = {
    0: "chicken",
    1: "cow",
    2: "creeper",
    3: "enderman",
    4: "pig",

```

```

5: "sheep",
6: "skeleton",
7: "spider",
8: "zombie"
}

# Creates DataFrame
df_classes = pd.DataFrame({
    "Class": [class_names[i] for i in sorted(class_counts.keys())],
    "Annotations": [class_counts[i] for i in sorted(class_counts.keys())]
})
#Prints the data frame
print(df_classes)

```

Show hidden output

Section 3: Model initialisation and training

Both the Nano (nano = YOLO("yolo11n.pt")) and Small (small = YOLO("yolo11s.pt")) models were initialised. Training involved specifying the dataset locations, the image size, and the batch size. The number of epochs, representing the number of training cycles, was adjusted throughout the process. Initially, the models were trained for ten epochs before this value was increased to fifty epochs. Pre-trained models were selected to enable a comparison between a larger, more comprehensive model and a smaller, faster alternative.

```

#####
# Section 3 : Model implementation and training
#####

# Initialise model: YOLO11n nano
nano = YOLO("yolo11n.pt")
#Trains the nano model
nano.train(
    data="data.yaml",
    epochs=50,
    imgsz=640,
    batch=16,
)
# Initialise model: YOLO11s small
small = YOLO("yolo11s.pt")
# Trains the small model
small.train(
    data="data.yaml",
    epochs=50,
    imgsz=640,
    batch=16,
)

```

Show hidden output

Section 4 : Model evaluation

The evaluation begins by loading the best weights from the training phase for both models. The "best" weights refer to those saved from the epoch where the model achieved its highest performance on the validation set, rather than solely relying on the final epoch's weights. These models were subsequently tested using a rigorous approach with the 20% validation dataset previously set aside. Crucially, the models were prevented from any further learning or weight adjustments during this stage; their sole function was to generate predictions for accuracy assessment. To quantify the accuracy of these models, four key metrics were employed: precision, recall, mAP50, and mAP50-95. These extracted metrics were then compiled into a comparative data frame. This data frame is subsequently presented and exported as a spreadsheet file named final_50_Epoch_Comparison.csv.

```

#####
# Section 4: Model evaluation
#####

print("Extracting final metrics and training history...")

# 1. Grabs the final validation metrics where he model performed best(keeps your terminal printout working!)
nano_metrics = YOLO("/content/runs/detect/train/weights/best.pt").val(split="val")
small_metrics = YOLO("/content/runs/detect/train-2/weights/best.pt").val(split="val")

# Extract inference speeds
nano_speed_ms = nano_metrics.speed['inference']
small_speed_ms = small_metrics.speed['inference']

# Generate and print the data frame summary
comparison = pd.DataFrame({

```

```

"Model": ["YOLO11 Nano", "YOLO11 Small"],
"Precision": [nano_metrics.box.mp, small_metrics.box.mp],
"Recall": [nano_metrics.box.mr, small_metrics.box.mr],
"mAP50": [nano_metrics.box.map50, small_metrics.box.map50],
"mAP50-95": [nano_metrics.box.map, small_metrics.box.map],
"Avg. Inference Time (ms)": [nano_speed_ms, small_speed_ms],
"FPS": [1000 / nano_speed_ms, 1000 / small_speed_ms],
"Training Time (50 epochs)": ["7.9 mins", "8.1 mins"]
})

print("\n=== FINAL MODEL PERFORMANCE COMPARISON ===")
print(comparison)
comparison.to_csv("final_50_epoch_comparison.csv", index=False)

```

Show hidden output

Section 5 : Model Analysis

The analysis of the results begins with loading the training results from both models. A line graph is then created using Matplotlib to serve as a convergence curve. This graph visually represents the learning speed of each model and indicates whether their performance plateaued or continued to improve up to epoch 50. The resulting graph is saved as model_accuracy_line_graph.png. Finally, a bar chart is generated to display the final mAP50 scores from the evaluation section. These scores are presented side-by-side using green and blue bars, offering a clear visual comparison of the models' final accuracies.

```

#####
#Section 5: Analysis of results
#####

# Plot validation accuracy (mAP50) over time
#This loads the training results for both Models
nano_csv_path = "/content/runs/detect/train/results.csv"
small_csv_path = "/content/runs/detect/train-2/results.csv"

if os.path.exists(nano_csv_path) and os.path.exists(small_csv_path):
    # Load history files
    df_nano = pd.read_csv(nano_csv_path)
    df_small = pd.read_csv(small_csv_path)

    # Clean up column names
    df_nano.columns = df_nano.columns.str.strip()
    df_small.columns = df_small.columns.str.strip()

    # Set up the line graph layout
    plt.figure(figsize=(10, 6))

    # Plot Nano Curve (Solid Green Line)
    plt.plot(df_nano['epoch'], df_nano['metrics/mAP50(B)'],
             label='YOLO11 Nano', color='#4CAF50', linewidth=2.5, linestyle='-')

    # Plot Small Curve (Dashed Blue Line)
    plt.plot(df_small['epoch'], df_small['metrics/mAP50(B)'],
             label='YOLO11 Small', color='#2196F3', linewidth=2.5, linestyle='--')

    # Style the graph
    plt.title("Model Convergence: Validation Accuracy (mAP50) over 50 Epochs", fontsize=14, fontweight='bold', pad=15)
    plt.xlabel("Epochs", fontsize=12)
    plt.ylabel("Validation Accuracy (mAP50 Score)", fontsize=12)

    plt.xlim(1, 50)
    plt.ylim(0, 1.05)
    plt.grid(True, linestyle='--', alpha=0.5)
    plt.legend(fontsize=12, loc='lower right')
    plt.show() # Display the line graph

    # Save a file for downloading
    plt.savefig("model_accuracy_line_graph.png", dpi=300, bbox_inches='tight')
    print("\n Line graph generated successfully as 'model_accuracy_line_graph.png'!")
else:
    print("\n Error: Could not find 'results.csv' in /train/ or /train-2/ folders.")

# Plot the mAP50 performance side-by-side
plt.figure(figsize=(6, 4))
plt.bar(comparison["Model"], comparison["mAP50"], color=['#4CAF50', '#2196F3'])
plt.title("Minecraft Mob Detection: 50-Epoch Model Comparison (mAP50)")
plt.ylabel("mAP50 Score")
plt.ylim(0, 1.0)
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.show()

```

[Show hidden output](#)

Section 6 : JPG Minecraft mob test

This section tests the trained Models by passing a standalone JPG image through them to verify inference capabilities. The process begins by defining the target file path and reinitialising both the Nano and Small models using their best.pt weight files. An inference pipeline is then executed independently for both architectures. This step applies a confidence threshold of 25% to filter out uncertain predictions. When a mob is recognised above this threshold, the model plots a localised bounding box directly onto the image, accompanied by a text label and confidence percentage. Finally, as prediction outputs are saved to dynamically generated directories, the script locates the newly annotated files and displays them side-by-side, providing a visual comparison of the detections produced by each model.

```
#####
# Section 6: jpg test using a Minecraft mob
#####

# 1. Path to the uploaded JPG image inside the content folder
image_path = "/content/test_image.jpg"

# Load both models with clear names using their correct save paths
small_model = YOLO("/content/runs/detect/train-2/weights/best.pt")
nano_model = YOLO("/content/runs/detect/train/weights/best.pt")

print(f"Analyzing '{os.path.basename(image_path)}' for Minecraft mobs with both models...")
# Run prediction with the nano_model and save visually
results_nano = nano_model.predict(source=image_path, conf=0.25, save=True)

# Grab the output paths dynamically for the nano model
saved_dir_nano = results_nano[0].save_dir
base_name_nano = os.path.basename(results_nano[0].path)
output_visual_path_nano = os.path.join(saved_dir_nano, base_name_nano)

print("\n DETECTION COMPLETE (YOLO11 Nano)! Here is what the Nano Model found:")
display(Image(filename=output_visual_path_nano))
# Run prediction with the small_model and save visually
results_small = small_model.predict(source=image_path, conf=0.25, save=True)

# Grab the output paths dynamically for the small model
saved_dir_small = results_small[0].save_dir
base_name_small = os.path.basename(results_small[0].path)
output_visual_path_small = os.path.join(saved_dir_small, base_name_small)

print("\n DETECTION COMPLETE (YOLO11 Small)! Here is what the Small Model found:")
display(Image(filename=output_visual_path_small))
```

Analyzing 'test_image.jpg' for Minecraft mobs with both models...

image 1/1 /content/test_image.jpg: 640x640 (no detections), 16.1ms
 Speed: 3.8ms preprocess, 16.1ms inference, 1.0ms postprocess per image at shape (1, 3, 640, 640)
 Results saved to /content/runs/detect/predict

DETECTION COMPLETE (YOLO11 Nano)! Here is what the Nano Model found:



image 1/1 /content/test_image.jpg: 640x640 1 creeper, 16.5ms
 Speed: 5.2ms preprocess, 16.5ms inference, 9.1ms postprocess per image at shape (1, 3, 640, 640)
 Results saved to /content/runs/detect/predict-2

DETECTION COMPLETE (YOLO11 Small)! Here is what the Small Model found:



Results

Examining the model convergence graph reveals several notable trends. The YOLO11 Nano model exhibits a more gradual, almost exponential increase in validation mAP@50 during the early stages of training compared with the YOLO11 Small model. Despite this difference in learning progression, both models achieve similar validation scores by approximately epoch 15. Around epochs 18–19, the Nano model experiences a temporary decline in validation performance before recovering rapidly and continuing to improve until it closely matches the Small model. This brief fluctuation is likely attributable to the stochastic nature of the optimisation process rather than a sustained reduction in performance. Both models subsequently stabilise between epochs 40 and 50, indicating that convergence has been reached and that further training would likely yield only marginal improvements. The YOLO11 small model follows a slightly different learning trajectory. It demonstrates rapid improvement during the initial epochs, followed by a brief dip before continuing its upward trend. Throughout much of the training process, both models maintain comparable performance; however, from approximately epochs 32 to 35 onwards, the small model consistently achieves slightly higher validation mAP@50. Although this advantage persists until the end of training, the overall difference between the two models remains relatively small. Now, if we look at the Minecraft mob detection model comparison, what we can see is that the difference between the two models' mean average precision with an IoU threshold of 0.5 is visually a minute difference between the two bar charts. This suggests that, at least when it comes to the mean average precision with an IoU threshold of 0.5, they are very, very similar.

```
# Downloads jpg Annotated graphs from github
!wget -O Model_convergence_line_graph.jpg "https://raw.githubusercontent.com/Magistrate-dot/ML-AI-project-_Minecraft_Object_Detection/main/Model_convergence_line_graph.jpg"

!wget -O Box_chart_comparison.jpg "https://raw.githubusercontent.com/Magistrate-dot/ML-AI-project-_Minecraft_Object_Detection/main/Box_chart_comparison.jpg"

#Displays Annotated Graphs
#display(Image('/content/Model_convergence_line_graph.jpg'))

#display(Image('/content/Box_chart_comparison.jpg'))

import matplotlib.pyplot as plt
import matplotlib.image as mpimg

# List of your downloaded image paths
images = [
    '/content/Model_convergence_line_graph.jpg',
    '/content/Box_chart_comparison.jpg'
]

for img_path in images:
```

```
plt.figure(figsize=(10, 6)) # Adjust width/height as needed for your PDF layout
img = mpimg.imread(img_path)
plt.imshow(img)
plt.axis('off') # Hides the chart axis ticks/numbers
plt.show()
```

```
--2026-06-26 18:43:56-- https://raw.githubusercontent.com/Magistrate-dot/ML-AI-project-Minecraft_Object_Detection/main/Moc
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 65903 (64K) [image/jpeg]
Saving to: 'Model_convergence_line_graph.jpg'
```

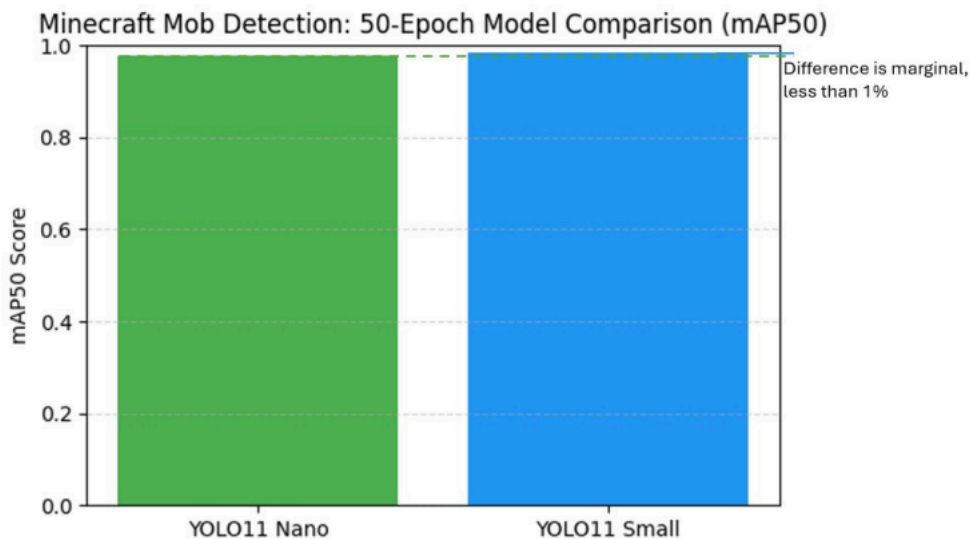
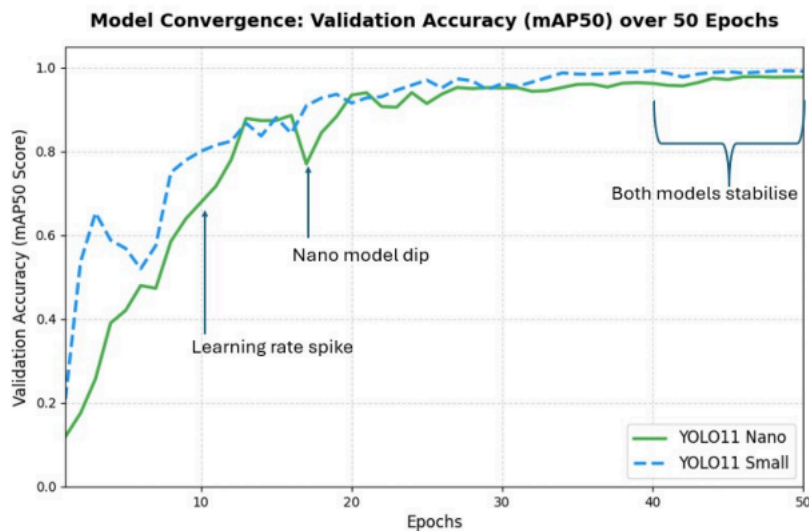
```
Model_convergence_l 100%[=====>] 64.36K --KB/s in 0.01s
```

```
2026-06-26 18:43:56 (5.41 MB/s) - 'Model_convergence_line_graph.jpg' saved [65903/65903]
```

```
--2026-06-26 18:43:56-- https://raw.githubusercontent.com/Magistrate-dot/ML-AI-project-Minecraft_Object_Detection/main/Box
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.111.133, 185.199.108.133, 185.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.111.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 60609 (59K) [image/jpeg]
Saving to: 'Box_chart_comparison.jpg'
```

```
Box_chart_compariso 100%[=====>] 59.19K --KB/s in 0.01s
```

```
2026-06-26 18:43:57 (4.94 MB/s) - 'Box_chart_comparison.jpg' saved [60609/60609]
```



To provide an overall comparison between the two models, the following performance metrics were evaluated: precision, recall, mAP50, mAP50-95, average inference time (ms), frames per second (FPS), and training time.

Examining the precision scores, the YOLO11 Small model achieved a slightly higher precision (0.969) than the YOLO11 Nano model (0.951). This indicates that when the Small model predicts the presence of a Minecraft mob, it is more likely to be correct, resulting in fewer false positive detections.

Interestingly, the opposite trend is observed for recall. The YOLO11 Nano model achieved a higher recall score (0.933) compared with the YOLO11 Small model (0.900). This suggests that the Nano model successfully detects a greater proportion of the true Minecraft mobs within the dataset, meaning it is less likely to miss an object during detection.

Looking at the mAP50 scores, both models achieved excellent overall detection performance. The YOLO11 Nano model obtained a score of 0.979, while the YOLO11 Small model achieved 0.984. Although the difference between the two models is only approximately 0.006 mAP (around 0.6 percentage points), the Small model demonstrates slightly stronger overall detection performance when evaluated at an IoU threshold of 0.5.

A similar trend can be observed for mAP50-95. The Nano model achieved a score of 0.795, whereas the Small model obtained 0.809. Since this metric evaluates performance across increasingly strict IoU thresholds, the results suggest that the YOLO11 Small model produces more accurately localised bounding boxes than the Nano model.

When considering computational efficiency, the YOLO11 Nano model demonstrates a clear advantage. The average inference time is 7.74 ms compared with 11.87 ms for the Small model, allowing the Nano model to process images approximately 35% faster. This is reflected in the FPS results, where the Nano model achieves approximately 129 FPS compared with 84 FPS for the Small model. Although the Nano model also completes training slightly faster (7.9 minutes compared with 8.1 minutes), the difference in training time is relatively small. Therefore, the primary efficiency advantage of the Nano model lies in its faster inference speed rather than its training time.

Overall, the results demonstrate a clear trade-off between detection accuracy and computational efficiency. The YOLO11 Small model achieves marginally higher precision, mAP50, and mAP50-95 scores, indicating superior overall detection and localisation performance. Conversely, the YOLO11 Nano model achieves a higher recall while offering significantly faster inference and a substantially higher frame rate. These findings suggest that although the Small model provides the highest overall accuracy, the Nano model offers a more computationally efficient solution while sacrificing only a small amount of detection performance.

Final Model Performance Comparison

Model	Precision	Recall	mAP50	mAP50-95	Avg. Inference Time (ms)	FPS	Training time (50 epochs)
YOLO11n	0.950623	0.933217	0.978581	0.795142	7.735268	129.278000	7.9 mins
YOLO11s	0.969365	0.899752	0.984374	0.808904	11.874292	84.215545	8.1 mins

To complement the quantitative evaluation, a qualitative inference test was performed using a previously unseen JPEG image containing multiple Minecraft mobs. Both trained models were evaluated using their optimal operational confidence threshold, identified from the F1–Confidence curve as the point that provides the best balance between precision and recall.

The results generally reflect the quantitative performance observed throughout the evaluation. The YOLO11 Small model successfully detected and produced a bounding box around the creeper, whereas the YOLO11 Nano model failed to identify this object. This observation is consistent with the Small model's slightly higher mAP50 and mAP50-95 scores, indicating superior overall detection and localisation performance.

It is also noteworthy that neither model detected the ocelot positioned behind the creeper. As the ocelot class was not included in the training dataset, this behaviour is expected and demonstrates that neither model incorrectly classified an unseen object as one of the trained classes. This suggests that both models generalise appropriately by avoiding false positive detections on unfamiliar objects.

Discussion

Within this section, the results presented previously are interpreted and considered in relation to the project's research objective. The primary finding of this study is that although two object detection models may achieve very similar quantitative performance metrics, their behaviour during practical deployment can still differ in meaningful ways. Although the YOLO11 Nano and YOLO11 Small models achieved comparable precision, recall, and mAP scores, the qualitative inference test highlighted a practical difference in detection performance. Specifically, the Nano model failed to detect the Creeper, whereas the Small model successfully identified and localised it.

This finding suggests that quantitative evaluation alone may not fully capture how a model performs in practical applications. Standard performance metrics remain essential for objectively comparing models; however, they should be complemented by qualitative testing to verify that the model behaves as expected when deployed on unseen data. In this project, the numerical differences between the two models were relatively small, yet these differences were reflected in the qualitative test, where the Small model demonstrated more reliable object detection.